

SZEGEDI TUDOMÁNYEGYETEM
Természettudományi és Informatikai Kar
Számítástudomány Alapjai Tanszék

Szakdolgozat

Közösségi sportesemény-szervező webalkalmazás tervezése és fejlesztése

Design and Development of a Community Sports Event Organizer Web
Application

Készítette:

Rák Dávid

programtervező informatikus
szakos hallgató

Témavezető:

Dr. Németh Tamás

egyetemi adjunktus

Szeged

2026

Feladatkiírás

Manapság az amatőr sportesemények szervezése gyakran széttagolt, és nagyrészt általános közösségi média csoportokra vagy üzenetküldő alkalmazásokra korlátozódik. Bár léteznek eseményszervező platformok, hiányzik egy olyan dedikált, átfogó webes megoldás, amely kifejezetten a helyi sportközösségeket célozza meg, és innovatív funkciókkal segíti a csapattársak vagy ellenfelek megtalálását.

A hallgató feladata egy olyan modern webalapú alkalmazás megtervezése és lefejlesztése, amely lehetővé teszi a helyi sportesemények egyszerű létrehozását, keresését és az azokra történő jelentkezést. A rendszernek két fő innovációs pillérre kell épülnie. Egyrészt geolokációs alapon, interaktív térképes felületen kell megjelenítenie a sportolási lehetőségeket, biztosítva a felhasználó saját pozíciójához viszonyított távolságalapú szűrést. Másrészt az alkalmazásnak tartalmaznia kell egy érdeklődésalapú ajánlórendszert, amely a rögzített felhasználói sportpreferenciák és a korábbi eseményeken való részvétel alapján személyre szabott javaslatokat tesz a közeli, releváns programokra.

A feladat kiterjed a teljes körű felhasználókezelés és a megbízható adatbázis-alapú tárolás megvalósítására is. A megoldás opcionálisan bővíthető további egyéni ötletekkel.

Tartalmi összefoglaló

- ***A téma megnevezése:***

Közösségi sportesemény-szervező webalkalmazás tervezése és fejlesztése

- ***A megadott feladat megfogalmazása:***

Helyi amatőr sportesemények (pl. futás, foci) szervezését, keresését és a jelentkezést támogató webalkalmazás fejlesztése, interaktív térképpel és személyre szabott ajánlórendszerrel.

- ***A megoldási mód:***

Kliens-szerver architektúrájú webes rendszer. A helyalapú keresést külső térkép integrálása, az ajánlásokat pedig a felhasználói preferenciák és részvételi adatok logikai összekapcsolása biztosítja.

- ***Alkalmazott eszközök, módszerek:***

A kliensoldal Angular keretrendszerben készült. A szerveroldali logika Python nyelven, Django és Django REST Framework segítségével került implementálásra, az adatok tárolását pedig PostgreSQL relációs adatbázis-kezelő végzi.

- ***Elért eredmények:***

Egy stabil, reszponzív platform, amely megbízhatóan kezeli a jelentkezéseket, pontosan szűr geolokáció alapján, és intelligensen ajánl eseményeket, hatékonyan támogatva a közösségek önszerveződését.

- ***Kulcsszavak:***

webalkalmazás, sportesemény-szervezés, geolokáció, ajánlórendszer, Angular, Django

Tartalomjegyzék

Feladatkiírás	1
Tartalmi összefoglaló	2
Bevezetés	4
1. Problémafelvetés és a meglévő megoldások vizsgálata	6
1.1. A közösségi sportesemény-szervezés kihívásai	6
1.2. Piaci körkép és hasonló célú alkalmazások elemzése	6
1.3. A geolokáció és a személyre szabott ajánlások létjogosultsága a témában	7
2. Alkalmazott technológiák bemutatása és kiválasztásuk okai	8
2.1. Kliensoldali környezet: Angular keretrendszer	8
2.2. Szerveroldali architektúra: Python, Django és Django REST Framework	8
2.3. Adatbázis-kezelés: PostgreSQL	9
2.4. Térképes szolgáltatások és geolokációs API-k (Google Maps)	10
3. A szoftver tervezésének folyamata	11
3.1. Követelményanalízis	11
3.2. Használati esetek	12
3.3. Rendszerarchitektúra és komponensdiagramok	13
3.4. Adatbázis-tervezés	13
3.5. Főbb folyamatok modellezése	14
4. A rendszer implementációja	16
4.1. Az adatbázis és a backend logika megvalósítása	16
4.2. Felhasználókezelés, hitelesítés és jogosultságok	16
4.3. A térképes megjelenítés és a távolságalapú szűrés integrációja	17
4.4. Az érdeklődésalapú ajánlórendszer algoritmusának implementálása	17
4.5. A frontend és backend kommunikációja	18
5. Az elkészült szoftver bemutatása	19
5.1. Regisztráció és a felhasználói sportpreferenciák rögzítése	19
5.2. Új sportesemények létrehozása és adminisztrációja	20
5.3. Interaktív eseménykeresés és térképi böngészés	20
5.4. Az ajánlórendszer működése és a javaslatok generálása a felületen	21
5.5. Csatlakozás az eseményekhez, értesítési rendszer és értékelés	22
6. Tesztelés és kiértékelés	25
6.1. Funkcionális és felhasználói elfogadási tesztelés	25
6.2. A geolokációs és ajánló algoritmus pontosságának vizsgálata	27
6.3. Ismert korlátok és továbbfejlesztési lehetőségek	28
Irodalomjegyzék	29
Nyilatkozat	30
Mellékletek	31

Bevezetés

Szakedolgozatomban olyan webes alkalmazás elkészítése volt a cél, amely valós, mindennapi problémára nyújt megoldást, és aktívan támogatja a közösségépítést. A rendszeres testmozgás és a csapatsportok mindig is közel álltak hozzám, így tapasztalatból tudom, hogy egy-egy amatőr sportesemény (például kispályás foci, kosárlabda vagy közös futás) megszervezése gyakran komoly nehézségekbe ütközik. Első lépésként megvizsgáltam a jelenleg elérhető platformokat és a felhasználói szokásokat. Arra kerestem a választ, hogy a meglévő megoldások mennyire hatékonyak a csapattársak vagy ellenfelek felkutatásában, és érdemes-e egy teljesen új, dedikált sportesemény-szervező platformot létrehozni.

A kutatás során azt tapasztaltam, hogy a szervezés manapság rendkívül széttagolt; leginkább általános közösségi média csoportokban vagy zárt üzenetküldő alkalmazásokban történik. Bár léteznek általános eseményszervező oldalak, olyan weblapú szoftvert, amely kifejezetten a helyi amatőr sportközösségekre fókuszál, interaktív térképen mutatja be a lehetőségeket, és még proaktívan ajánl is programokat, alig találni. A zárt csoportok miatt a környéken élők sokszor nem is szereznek tudomást a nyitott sportolási lehetőségekről.

A feladat egy olyan modern, kliens-szerver architektúrára épülő webalkalmazás megtervezése és lefejlesztése, amely egyetlen központi felületen teszi lehetővé a helyi sportesemények létrehozását, keresését és a jelentkezést. A rendszer célja, hogy a sportolni vágyók a böngészőből könnyen és gyorsan találjanak maguknak megfelelő mozgásformát. A megoldás alapját olyan nyílt forráskódú és modern technológiák biztosítják, mint az Angular és a Django keretrendszerek, amelyek stabil és gyors működést tesznek lehetővé.

A szoftver egyik legfőbb innovációs eleme a geolokációs technológiára épülő működés. Ebben a nézetben a felhasználók egy interaktív térképen láthatják a környezetükben zajló eseményeket. Lehetőség van a saját pozíció alapján történő „események a közelemben” funkció használatára és távolságszűrésre is, így mindenki azonnal átláthatja, hogy a lakhelye vagy munkahelye közelében milyen sportolási lehetőségek érhetők el. Az alkalmazás felületén nemcsak passzívan böngészni lehet, hanem a felhasználók maguk is néhány kattintással létrehozhatnak új eseményeket, pontos helyszínhez kötve azokat.

Az alkalmazás másik kiemelt részét egy érdeklődésalapú ajánlórendszer képezi. Regisztráció után a felhasználók rögzíthetik a saját sportpreferenciáikat, a rendszer pedig ezt, valamint az

eddiggi eseményeken való részvételüket alapul véve személyre szabott javaslatokat tesz. Az algoritmus összeköti a helyadatokat a felhasználói profilokkal, így képes arra, hogy célzottan a legrelevánsabb, közeli programokat ajánlja fel. Ezzel a funkcióval a szoftver aktívan segíti a hasonló érdeklődésű emberek egymásra találását és a helyi sportközösségek önszerveződését, kiküszöbölve a hagyományos szervezésből fakadó kommunikációs hiányosságokat.

1. Problémafelvetés és a meglévő megoldások vizsgálata

1.1. A közösségi sportesemény-szervezés kihívásai

Az amatőr csapatsportok – mint például a kispályás labdarúgás, a kosárlabda vagy a röplabda – egyik legnagyobb logisztikai kihívása a megfelelő létszám összegyűjtése. Egy mérkőzés megszervezése során a szervezőknek számos akadállyal kell szembenéznük: a játékosok váratlan lemondásaival, a megfelelő tudásszintű csapattársak vagy ellenfelek felkutatásával, valamint a helyszín és az időpont egyeztetésével.

A gyakorlatban ezek a szervezési folyamatok ma jellemzően zárt, elszigetelt csatornákon zajlanak. A leggyakoribb eszközök a Facebook csoportok, a Messenger, a Viber, és egyéb zárt üzenetküldő alkalmazások. Ezek a platformok bár alkalmasak a gyors kommunikációra egy már meglévő ismeretségi körön belül, jelentős korlátokkal rendelkeznek a közösségbővítés terén. Egy új városba költöző, vagy csak egy alkalmi mérkőzésre beugrani vágyó sportolni szerető egyén számára szinte lehetetlen rátalálni ezekre a zárt szerveződésekre. Emellett az üzenetfolyamokban könnyen elvesznek a lényeges információk, és a szervezőnek manuálisan kell vezetnie a résztvevők listáját. Ebből adódóan egyértelmű az igény egy olyan nyílt, transzparens és dedikált felületre, amely automatizálja a jelentkezések kezelését, és láthatóvá teszi az eseményeket a szélesebb, helyi közönség számára is.

1.2. Piaci körkép és hasonló célú alkalmazások elemzése

A weben és a mobilalkalmazás-áruházakban jelenleg is találhatók olyan szoftverek, amelyek az eseményszervezést hivatottak megkönnyíteni, azonban ezek legtöbbje nem nyújt teljes körű megoldást a fent vázolt problémára.

- Általános eseményszervező platformok (pl. Facebook Events, Meetup): Ezek a rendszerek kiválóak nagy létszámú, nyilvános események (koncertek, előadások) meghirdetésére, de hiányoznak belőlük a sport-specifikus funkciók. Nem képesek finomhangolt, sportág- és nehézségi szint alapú szűrésre, nem kezelik a sporteseményeknél kritikus (és a résztvevők számával dinamikusan változó) kapacitáskorlátokat, és a keresési funkcióik sem kifejezetten a gyors, lokális szűrésekre lettek optimalizálva.

- Pályafoglaló rendszerek: Számos sportlétesítmény rendelkezik saját online foglalási rendszerrel, ám ezek fókuszában a helyszín értékesítése áll, nem pedig a játékosok összekötése. Ha egy ötfős társaság lefoglal egy pályát, de hiányzik még öt ember a játékhoz, a pályafoglaló rendszer nem segít megtalálni őket.
- Dedikált sportalkalmazások: Léteznek külföldi piacra fókuszáló, professzionálisabb sport-menedzsment alkalmazások (pl. SportEasy), amelyek elsősorban állandó, versenyszerűen működő csapatok adminisztrációjára (tagdíjak, edzésnaplók) fókuszálnak, nem pedig az alkalmi, utcai jellegű, lazább szerveződésekre.

A piackutatás alapján megállapítható, hogy hiánytöltő lenne egy olyan hazai/helyi viszonyokra optimalizált webalkalmazás, amely az egyszerű és gyors, alkalmi sportesemények létrehozására és a játékosok helyalapú összekötésére koncentrál.

1.3. A geolokáció és a személyre szabott ajánlások létjogosultsága a témában

A vizsgált alternatívák hiányosságaira reagálva, a saját fejlesztésű webalkalmazásom két olyan technológiai megközelítést alkalmaz, amely radikálisan javítja a felhasználói élményt: a geolokációt és az érdeklődésalapú ajánlórendszert.

A geolokációs funkciók integrálása azért elengedhetetlen, mert az amatőr sportolás egy erősen helyhez kötött tevékenység. A felhasználókat elsősorban az érdekli, hogy a lakóhelyük vagy a munkahelyük közvetlen közelében, utazás nélkül milyen sportolási lehetőségek érhetők el. A hagyományos, listaalapú megjelenítés helyett egy interaktív térkép azonnali vizuális visszacsatolást ad az események térbeli elhelyezkedéséről, míg a távolságalapú szűrők segítségével a rendszer automatikusan kizárja a felhasználó számára irreleváns, túl messze lévő találatokat.

A személyre szabott ajánlórendszer implementálása egy további intelligens réteget ad a platformhoz. Ahogy a felhasználói bázis és az események száma növekszik, az információbőség problémájával szembesülhetünk: a túl sok esemény között a felhasználó nehezen találja meg a számára érdekeseket. Az algoritmus a regisztrált preferenciák kedvelt sportágak és a korábbi aktivitások elemzésével képes proaktívan a felhasználó elé tárni azokat az eseményeket, amelyekre a legnagyobb valószínűséggel jelentkezne. Ez nemcsak a keresésre fordított időt csökkenti, de jelentősen növeli a platformhoz való visszatérés hajlandóságát, ezáltal élénkítve a helyi sportközösség aktivitását.

2. Alkalmazott technológiák bemutatása és kiválasztásuk okai

2.1. Kliensoldali környezet: Angular keretrendszer

Az alkalmazás felhasználói felülete a Google által fejlesztett Angular [1] keretrendszer segítségével készült. Az Angular egy komponensalapú, nyílt forráskódú platform, amely az úgynevezett Single Page Application (SPA) architektúra megvalósítására fókuszál. Ennek lényege, hogy a böngésző csak egyszer tölti be az alapvető HTML, CSS és JavaScript [3] fájlokat, a további navigáció és adatlekérés pedig a háttérben, aszinkron módon történik, így az oldalújrátöltések elmaradása miatt a szoftver sokkal gyorsabb, asztali alkalmazásokhoz hasonló felhasználói élményt nyújt.

A kiválasztás okai: A piacon elérhető népszerű alternatívák (mint a React vagy a Vue.js) közül az Angular mellett több nyomós érv is szólt.

- **Típusbiztonság:** Az Angular alapértelmezetten a TypeScript [2] nyelvet használja, amely a JavaScript statikusan típusos kiterjesztése. A fordítási időben történő típusellenőrzés drasztikusan csökkenti a futásidejű hibák számát, és sokkal átláthatóbbá teszi az objektumok kezelését.
- **Keretrendszer-szintű integráció:** Míg más eszközök inkább csak részmegoldásokat kínáló könyvtárak, az Angular egy teljes körű keretrendszer. Beépítve tartalmazza a HTTP klienseket, a bonyolult űrlapkezelést (Reactive Forms) és a beépített útválasztást (Routing).
- **Adatfolyamok kezelése:** A geolokációs funkciók és az interaktív térképi elemek állapotának folyamatos frissítése megköveteli a komplex aszinkron események kezelését. Az Angular által használt Reactive Extensions for JavaScript (RxJS) könyvtár hatékony megoldást kínál ezekre a folyamatokra.

2.2. Szerveroldali architektúra: Python, Django és Django REST Framework

Az alkalmazás üzleti logikáját, az adatbázis-műveleteket, valamint az ajánlórendszer algoritmusait a szerveroldalon valósítottam meg. A fejlesztéshez a Python programozási nyelvet, azon belül pedig a Django keretrendszert [4] használtam.

A kliens és a szerver közötti kommunikációt egy RESTful API biztosítja, amelynek felépítéséhez a Django REST Framework [5] kiegészítést alkalmaztam. Az API szabványos JSON formátumban cserél adatot az Angular klienssel.

A kiválasztás okai:

- Teljes körű megoldás: A Django keretrendszer rengeteg beépített eszközt tartalmaz (például felhasználó-hitelesítés, jelszó-hashelés, adminisztrációs felület), ami lehetővé tette, hogy a redundáns alapfunkciók kódolása helyett az alkalmazás egyedi üzleti logikájára koncentrálhassak.
- Adatkezelési képességek: A Django beépített Object-Relational Mapping (ORM) rendszere magas szintű, Python-alapú absztrakciót biztosít az adatbázis felett, így a komplex SQL lekérdezések biztonságosabban és átláthatóbban írhatók meg.
- Algoritmikus támogatás: A személyre szabott ajánlórendszer működéséhez adatok elemzése és párosítása szükséges. A Python jelenleg az iparági standard az adatfeldolgozás és a gépi tanulás területén, így ha a jövőben az ajánlórendszer mesterséges intelligencia alapú modullal bővülne, a technológiai alapok már adottak.

2.3. Adatbázis-kezelés: PostgreSQL

Az adatok tartós és biztonságos tárolását a PostgreSQL relációs adatbázis-kezelő [6] látja el.

A kiválasztás okai:

- Integritás és megbízhatóság: A sportesemények szervezésekor kritikus a tranzakciók biztonsága (például egy adott eseményre ne jelentkezessenek többen, mint a maximális létszám). A PostgreSQL egy teljes mértékben ACID konform rendszer, amely garantálja az adatok konzisztenciáját.
- Térinformatikai kiterjeszthetőség: Bár az alapvető távolságszámítások matematikai képletekkel is elvégezhetők, a PostgreSQL – a PostGIS kiterjesztés révén – natív módon, a legmagasabb szinten támogatja a térbeli adatok (koordináták) tárolását és a geolokációs lekérdezéseket. Ez a funkcionalitás tökéletesen illeszkedik a platform "események a közelemben" koncepciójához.

2.4. Térképes szolgáltatások és geolokációs API-k (Google Maps)

Az alkalmazás vizuális térképi elemeinek megjelenítéséért, valamint a komplex helymeghatározási folyamatokért a Google Maps API ökoszisztémája felel. A rendszer a kliensoldalon közvetlen HTTP kéréseken keresztül kommunikál a Google felhőszolgáltatásaival, amelyhez két specifikus modult integráltam:

- Google Places API (New) [8]: Ez a szolgáltatás végzi a felhasználó által beírt keresőszavak (címek vagy létesítménynevek) koordinátává alakítását.
- Google Geocoding API [9]: Ez a szolgáltatás felel az inverz geokódolásért, amely a beolvasott GPS koordinátákat formázott, emberi olvasásra alkalmas lakcímmé alakítja át.

A kiválasztás okai:

- Kiemelkedő adatminőség és POI keresés: A Google rendelkezik a világ legkiterjedtebb és legfrissebb helyszín-adatbázissal. Az amatőr sportesemények szervezésekor kulcsfontosságú, hogy a felhasználók ne csak pontos postai címekre kereshessenek, hanem sportcsarnokok, közismert parkok vagy iskolai pályák neveire is. A Places API intelligens szövegkeresője ezt maradéktalanul biztosítja.
- Megbízható inverz geokódolás: A modern webes felhasználói élmény megköveteli, hogy a rendszer automatikusan fel tudja ismerni a felhasználó helyzetét. A Google Geocoding API rendkívül gyors és pontos válaszidővel képes a böngészőből kinyert "nyers" szélességi és hosszúsági fokokat utcanév szintű adatokká fordítani.
- Architektúrális illeszkedés és biztonság: A Google REST API-k aszinkron hívásai tökéletesen illeszkednek a keretrendszer adatfolyam-kezelésébe. Továbbá a közvetlen API hívások lehetővé tették egy olyan biztonságos hálózati réteg kialakítását, amely elkülöníti a külső Google kéréseket a belső hitelesített kérésektől.

3. A szoftver tervezésének folyamata

A szoftverfejlesztés egyik legkritikusabb szakasza az implementációt megelőző alapos rendszertervezés. Jelen fejezet célja, hogy bemutassa az alkalmazás mögött meghúzódó tervezési döntéseket, a követelmények specifikálásától kezdve az architektúráis felépítésen át egészen az adatbázis és a főbb folyamatok modellezéséig. A tervezés és dokumentálás során a szoftverfejlesztésben standardnak számító Unified Modeling Language (UML) eszköztárát alkalmaztam.

3.1. Követelményanalízis

A fejlesztés megkezdése előtt pontosan definiálni kellett, hogy a platformnak milyen feladatokat kell ellátnia, és milyen minőségi elvárásoknak kell megfelelnie.

Funkcionális követelmények:

- **Felhasználókezelés és profilozás:** A rendszer tegye lehetővé a felhasználók regisztrációját, bejelentkezését (JWT token alapú hitelesítéssel), valamint saját profiljuk testreszabását, adataik változtatását. A profilnak tartalmaznia kell a földrajzi alapértelmezéseket (alapértelmezett helyszín és keresési sugár), a sportpreferenciákat illetve a felhasználó korábbi tevékenységeit.
- **Eseménykezelés:** A felhasználók hozhassanak létre új sporteseményeket a cím, a sportág, a leírás, a nehézségi szint, a pontos dátum, az időtartam, a helyszín és a kapacitás (minimum és maximum létszám, valamint fenntartott helyek) megadásával. Az események lehetnek publikusak, köthetők előzetes szervezői jóváhagyáshoz, illetve beállítható, hogy ingyenesek vagy díjhoz kötöttek legyenek (fedezve ezzel az esetleges pályabérleti vagy egyéb költségeket). A szoftvernek mindezek mellett garantálnia kell a saját események utólagos szerkesztésének, illetve lemondásának lehetőségét is.
- **Jelentkezés és létszámkezelés:** A rendszer biztosítsa a nyitott eseményekre történő jelentkezést. Dinamikusan számolja a résztvevőket (beleértve az alkalmazáson kívüli "plusz vendégeket" is), és akadályozza meg a túljelentkezést.
- **Értesítési rendszer:** A platform küldjön belső illetve emailes értesítéseket a felhasználóknak a releváns interakciókról (pl. új jelentkezés egy saját eseményre, jelentkezés jóváhagyása vagy elutasítása, új értékelés, ajánlott esemény létrejötte).

- Térképes keresés és ajánlórendszer: Az elérhető események egy interaktív térképen jelennek meg, segítve a földrajzi alapú tájékozódást. Míg a térkép a lokációk szemléltetésére szolgál, az ajánlórendszer a háttérben rangsorolja a találatokat, biztosítva, hogy a kapcsolódó listanézetben a felhasználó számára legrelevánsabb események kerüljenek előtérbe.
- Értékelési rendszer: Az események kezdete után a résztvevők 1-től 5-ig terjedő skálán értékelhessék az eseményt és szöveges visszajelzést hagyhassanak.

Nem funkcionális követelmények:

- Kliensoldali reszponzivitás: A felületnek Single Page Application architektúrában kell működnie, garantálva a gyors oldalváltásokat és az asztali alkalmazásokhoz hasonló sebességet.
- Adatintegritás és biztonság: A jelszavakat titkosítva kell tárolni. A párhuzamos jelentkezésekből fakadó inkonzisztenciákat adatbázis-szintű kényszerekkel kell kivédeni.
- Típusbiztonság: A frontend és backend kommunikációja során szigorú adattípusokat kell használni az adatok konzisztenciája érdekében.

3.2. Használati esetek

A követelmények alapján a rendszerben alapvetően négy fő jogosultsági kört különíthetünk el, amelyek egymásra épülnek:

1. Látogató (Nem hitelesített felhasználó): Korlátozott jogosultságokkal rendelkezik. A főoldalon kedvcsináló jelleggel látja a publikus, közelgő eseményeket, de a részletek megtekintéséhez vagy interakcióhoz be kell lépnie vagy regisztrálnia kell.
2. Regisztrált felhasználó (Sportoló): A rendszer alapszereplője. Böngészhet az események között, beállíthatja a preferenciáit, csatlakozhat eseményekhez, értékelhet eseményeket, módosíthatja a profilját, és olvashatja a neki szóló értesítéseket.
3. Szervező: Olyan regisztrált felhasználó, aki a "Sportesemény létrehozása" funkciót aktiválja. Jogosultsága van a saját eseménye adatainak szerkesztésére, a jelentkezők kezelésére (elfogadás, elutasítás), valamint az esemény lemondására.

4. Adminisztrátor (Rendszergazda): A legmagasabb szintű jogosultsággal rendelkező szereplő. Nem a kliensoldali Angular felületet, hanem egy elkülönített, védett adminisztrációs panelt használ. Feladata a platform háttéradatainak karbantartása: kezeli a regisztrált felhasználókat, moderálhatja a nem megfelelő eseményeket, valamint karbantartja a referenciatáblákat.

3.3. Rendszerarchitektúra és komponensdiagramok

A webalkalmazás egy rétegzett, RESTful kliens-szerver architektúrára épül, amely logikailag és fizikailag is elkülöníti a felületet az üzleti logikától:

- Frontend (Angular Kliens): TypeScript nyelven írt, böngészőben futó komponens. A modell rétege a szerver válaszait típusos interfészekké (pl. `SportEvent`, `User`, `PaginatedResponse`) alakítja, így már fordítási időben biztosítja a hibamentes adatkezelést.
- Backend (Django REST Framework API): Python alapú kiszolgáló réteg. Felelős az adatok validációjáért (serializer-ek segítségével), az ajánló algoritmus futtatásáért, a létszámkorlátok szerveroldali ellenőrzéséért, és a jogosultságkezelésért.
- Perzisztencia réteg (PostgreSQL): Relációs adatbázis-kezelő, amely tranzakcióbiztosan tárolja a rendszer állapotát.
- Külső API-k: A geokódolásért és a térképi felületért a Google Maps szolgáltatásai felelnek, amelyekkel az Angular kliens aszinkron hívásokon keresztül kommunikál.

3.4. Adatbázis-tervezés

Az adatbázis struktúrája a harmadik normálforma (3NF) szabályait követi, minimalizálva az adatredundanciát és garantálva az adatmódosítási anomáliák elkerülését. A rendszer a következő főbb entitásokból épül fel:

- User (Felhasználó): A Django beépített felhasználói modelljének kiterjesztése, amely a hitelesítési adatokon túl a személyes profilinformációkat és a geolokációs alapértelmezéseket (preferált helyszín és keresési sugár) is tárolja.
- SportType (Sportág) és UserSportPreference: A *SportType* egy statikus szótártábla a támogatott sportágakról, míg a *UserSportPreference* egy kapcsolótábla a felhasználó

és a sportág között. Ez utóbbi entitás tárolja a felhasználó tudásszintjét és érdeklődési szintjét, amelyek az ajánlórendszer kulcsbemeneteiként szolgálnak.

- **SportEvent (Esemény):** A platform központi entitása, amely a követelményeknél definiált összes alapalkotót (cím, sportág, leírás, árazás) magába foglalja. Adatbázis-technikai szempontjából kiemelendő, hogy a helyszíneket a térképes integrációhoz elengedhetetlen pontos földrajzi koordinátákkal rögzíti. Dinamikus attribútumai (például az *is_full* vagy az *available_spots*) számított mezőként üzemelnek: valós időben összesítik a visszaigazolt jelentkezéseket, az alkalmazáson kívüli plusz vendégeket (*extra_guests*) és a szervező által előzetesen lefoglalt helyeket (*reserved_spots*).
- **EventParticipant (Résztevő):** Kapcsolótábla a felhasználó és az esemény között. Egy beépített állapotgép (pending, confirmed, cancelled, rejected státuszokkal) segítségével követi le a jelentkezés teljes életciklusát, továbbá dedikált mezőkkel biztosít helyet az eseményt követő visszamenőleges értékeléseknek (*rating* és *feedback*).
- **Notification (Értesítés):** Aszinkron platformeseményekhez kötődő entitás, amely egy adott *User*-hez, mint címzetthez rendel különböző típusú (például új jelentkezés, jóváhagyás, új értékelés) rendszerüzeneteket.
- **EventImage (Képek):** *Megjegyzés: A szoftvertervezés ezen fázisában az adatbázisséma felkészült az eseményekhez tartozó többképes galériák tárolására, azonban a funkció tényleges kliensoldali implementációja a jelenlegi fejlesztési iterációnak nem része, az a későbbi továbbfejlesztési tervek között szerepel.*

3.5. Főbb folyamatok modellezése

A komponensek közötti kommunikáció megértéséhez kiváló példa a Jóváhagyást igénylő eseményre történő jelentkezés szekvenciája:

1. **Jelentkezés indítása:** A felhasználó az Angular felületen rákattint a "Jelentkezés" gombra egy *requires_approval=True* attribútummal rendelkező eseménynél. Opcionálisan megadhat üzenetet és plusz vendégeinek számát.
2. **Kérés a Backendhez:** A Kliens egy POST kérést (*/api/events/{id}/join/*) küld a Backendnek, mellékelve a felhasználó hitelesítő JWT tokenjét és az adatokat (pl. *JoinEvent payload*).

3. Validáció: A Backend ellenőrzi az üzleti logikát (pl. a *can_user_join* függvény alapján van-e még szabad hely, az esemény nem járt-e le, és a felhasználó nem jelentkezett-e már).
4. Adatbázis tranzakció: A rendszer létrehoz egy új *EventParticipant* rekordot *status='pending'* értékkel. Ezzel egy időben legenerál egy *Notification* rekordot is (*notification_type='join_request'*) az esemény Szervezőjének. A Backend 200-as OK státusszal válaszol.
5. Jóváhagyás: A Szervező később a felületen lekéri a függőben lévő kéréseit, és elfogadja a felhasználó jelentkezését. A Kliens egy PUT kérést (*UpdateParticipantStatus*) küld *status='confirmed'* értékkel.
6. Értesítés generálása: A Backend frissíti a résztvevő státuszát, és újabb *Notification* rekordot hoz létre (*join_approved*), ezúttal a felhasználó számára. Így a folyamat bezárul, és a dinamikus létszámlimit (*participants_count*) is frissül az adatbázisban.

4. A rendszer implementációja

4.1. Az adatbázis és a backend logika megvalósítása

A szerveroldali üzleti logika a Django REST Framework osztályalapú nézeteire és szerializálókra épül. Ezek az eszközök hatékony absztrakciót biztosítanak a PostgreSQL adatbázis felett.

A szoftver gerincét adó eseménykezelés a *events/views.py* fájlban valósult meg. Az események lekérdezését és létrehozását a *SportEventListCreateView* végzi, amely a *DjangoFilterBackend* segítségével támogatja az összetett paraméteres szűrést (pl. sportág, státusz, ingyenesség alapján). Az adatok validációjáért és JSON formátumba alakításáért a szerializálók felelnek. A *SportEventCreateSerializer* osztály komplex, objektumszintű ellenőrzéseket hajt végre a mentés előtt:

- Ellenőrzi, hogy a kezdési időpont nem esik-e a múltba.
- Kalkulálja a befejezési időpontot a megadott időtartam (*duration_minutes*) alapján.
- Biztosítja a logikai konzisztenciát (a minimum résztvevők száma nem haladhatja meg a maximumot, a fizetős eseményeknél pedig kötelező az ár megadni). Az adatbázisba történő írás során a DRF tranzakciókezelése garantálja az ACID tulajdonságok érvényesülését.

4.2. Felhasználókezelés, hitelesítés és jogosultságok

A rendszer egy stateless JWT alapú hitelesítési mechanizmust használ, amely ideális az Angular alapú Single Page Application architektúrákhoz.

A kliensoldalon a folyamatot egy Angular Interceptor vezérli. Ez az összetevő minden kimenő HTTP kérést elfog, és a fejlécbe illeszti az *Authorization: Bearer <token>* azonosítót. A robusztus működés érdekében az interceptor beépített hibakezeléssel rendelkezik: amennyiben a szerver 401-es (Unauthorized) hibakóddal válaszol (például a token lejáratára miatt), a kliens aszinkron módon megpróbálja frissíteni azt a refresh token segítségével. Ha ez sikertelen, a felhasználót automatikusan kijelentkezteti és átirányítja a bejelentkezési képernyőre.

A szerveroldalon a jogosultságokat a beépített és egyedi engedélyezési osztályok szabályozzák. Míg az események böngészéséhez elég a standard *IsAuthenticated* vagy *AllowAny* jogosultság, az események módosítását és törlését egy egyedi, *IsEventCreatorOrReadOnly* nevű

objektumszintű jogosultságkezelő osztály védi, amely biztosítja, hogy a módosításokat kizárólag a szervező hajthassa végre.

4.3. A térképes megjelenítés és a távolságalapú szűrés integrációja

A geolokációs funkciók vizuális megjelenítéséért a kliensoldalon az Angular rendszerbe integrált, nyílt forráskódú Leaflet [7] térképes könyvtár felel. A rendszer az OpenStreetMap csempéit használja a háttértérkép rendereléséhez.

A pontos koordináták kinyerése több módon történhet a felületen:

1. HTML5 Geolocation API: Az új esemény létrehozásakor a rendszer képes lekérdezni a felhasználó aktuális pozícióját a böngészőtől.
2. Geokódolás: A *GeocodingService* egy aszinkron HTTP kliensen keresztül kommunikál a Google Places API-val. A külső szolgáltató felé indított kéréseknél a natív *HttpBackend* osztályt alkalmaztam a standard *HttpClient* helyett. Ezzel a kérés elkerüli az alkalmazás globális *AuthInterceptor*-át, így a rendszer saját belső JWT tokenjei nem szivárognak ki a Google szerverei felé.

Amikor a frontend elküldi a koordinátákat, a backend végzi el a távolságalapú szűrést. Mivel a Föld felülete gömbszerű, a két koordináta közötti valós távolság kiszámításához az alkalmazás a Haversine-formulát [10] használja. A *calculate_distance* függvény a bekért szélességi és hosszúsági fokokat radiánná alakítja, majd a gömbi trigonometria szabályai alapján adja vissza a távolságot kilométerben. A rendszer ezen érték alapján szűkíti le az események listáját a felhasználó által meghatározott sugarú körön belülre.

4.4. Az érdeklődésalapú ajánlórendszer algoritmusának implementálása

A szoftver egyik legfőbb innovációja a *recommendation_service.py* fájlban megvalósított súlyozott ajánló algoritmus. A rendszer célja, hogy az információtúterheltség elkerülése végett mindig a legrelevánsabb eseményeket kínálja fel a főoldalon.

Az algoritmus az alábbi paramétereket integrálja egy dinamikusan kiszámított, normalizált pontszámba:

1. Preferenciák és Tudásszint: A rendszer megvizsgálja a profilban beállított érdeklődési szintet (1–10 pont). Ezt kiegészíti a nehézségi szint egyezésének elemzése: ha a felhasználó "tudásszintje" megegyezik az esemény "nehézségével", extra +3 pont jár,

szomszédos szint esetén +1 pont, míg nagy eltérésnél (pl. kezdő jelentkezne profi eseményre) nem jár pont.

2. Részvételi előzmények: A *get_participation_history_scores* metódus elemzi a múltbeli aktivitást. A sikeresen megerősített részvételek (+3 pont) és a pozitív (4 vagy 5 csillagos) utólagos értékelések bónusz pontokat generálnak az adott sportágra. A lemondott jelentkezések enyhe (-1 pont) büntetést kapnak.
3. Távolság alapú súlyozás: A már említett Haversine-formulával az algoritmus meghatározza az esemény és a felhasználó által beállított alapértelmezett helyszín közötti távolságot, melyből inverz módon képez 0 és 5 pont közötti értéket (minél közelebb van, annál több pont).
4. Teltségi szint: Az algoritmus előnyben részesíti a hamarosan betelő eseményeket. Ha a résztvevők száma eléri a kapacitás 50%-át, extra pont (+1), ha a 80%-át, akkor kiemelt bónusz (+2) adódik a végeredményhez, ösztönözve a gyors jelentkezést.

A pontozás után a rendezett lista (Top 20 esemény) kerül aszinkron módon visszaküldésre az Angular felület felé. Továbbá egy esemény létrehozásakor a szerver a háttérben azonnal lefuttatja a párosítást, és a *Notification* modellen keresztül értesítést generál azoknak, akiknek a preferenciájába tökéletesen illeszkedik az új esemény.

4.5. A frontend és backend kommunikációja

A két független komponens közötti adatcsere szabványos JSON formátumú REST API-n keresztül zajlik. Az Angular oldalon a kommunikációt az *EventService* menedzseli a megfigyelhető adatfolyamok segítségével.

Az adatok áramlása és transzformációja szigorúan típusos objektumokon keresztül történik, mint például a *SportEvent*, a *CreateSportEvent*. Az összetett lekérdezéseknél (mint a térképes keresés) a szolgáltatás az Angular *HttpParams* osztályát használja, amely dinamikusan fűzi a kulcs-érték párokat (például *user_lat*, *user_lng*, *radius*) az URL végére, kiszűrve a null vagy definiálatlan mezőket.

A válaszok aszinkron módon érkeznek vissza a komponensekhez, ahol a komponens logikája feldolgozza a HTTP státusz kódokat, frissíti az alkalmazás állapotát, és a *ToastService* segítségével vizuális visszajelzést (siker vagy form validációs hiba) nyújt a felhasználónak.

5. Az elkészült szoftver bemutatása

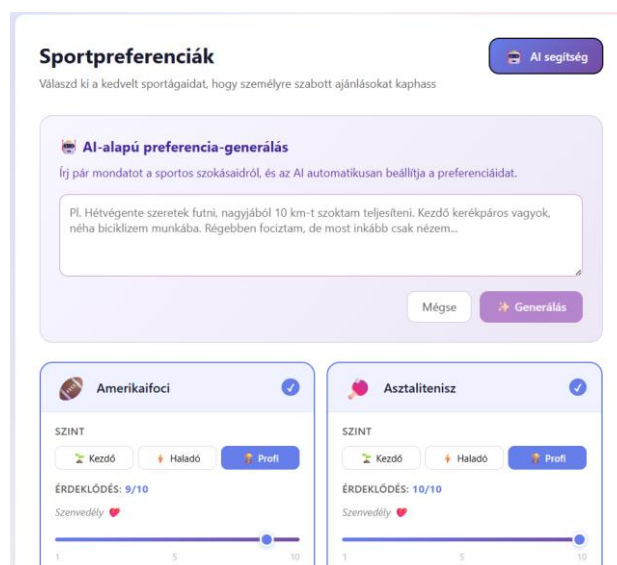
5.1. Regisztráció és a felhasználói sportpreferenciák rögzítése

Az alkalmazás használatának első lépése a regisztráció, majd a személyes profil és a sportpreferenciák beállítása. A preferenciák rögzítése kritikus fontosságú, hiszen ez képezi az ajánlórendszer alapját.

A profilon belül a "Sportág preferenciák szerkesztése" résznél a felhasználó egy vizuálisan vonzó, ikonokkal ellátott kártyás rácsszerkezetből választhatja ki az általa támogatott sportágakat. Egy sportág kiválasztásakor dinamikusan lenyílik a beállító panel, ahol két értéket kell megadni:

1. Tudásszint: Kezdő, Haladó vagy Profi gombokkal.
2. Érdeklődési szint: Egy 1-től 10-ig terjedő csúszka segítségével, amelyhez a rendszer azonnali szöveges visszacsatolást ad (pl. "Közepes" vagy "Szenvedély").

Az alkalmazás egyik kiemelt, kényelmi innovációja az AI-alapú preferencia-generálás. A fejlécben található "AI segítség" gombra kattintva lenyílik egy szövegdoboz, ahová a felhasználó szabad szöveges formában írhat a sportolási szokásairól (például: *"Hétvégeente szeretek futni, de néha beugrom a barátokkal focizni is"*). Az "Elemzés" gomb megnyomásával a háttérrendszer feldolgozza a szöveget, és automatikusan javaslatokat tesz a sportágakra, a szintre és az érdeklődés mértékére, amelyeket a felhasználó egyetlen kattintással elfogadhat, véglegesíthet.



5.1 ábra: Preferenciák beállítása

5.2. Új sportesemények létrehozása és adminisztrációja

Az eseményszervezés folyamatát egy letisztult, validációkkal ellátott űrlap támogatja. A szervező megadhatja az esemény alapadatait (cím, leírás, sportág, nehézségi szint), az időpontot, a helyszínt, a kapacitást, valamint egyéb beállításokat is alkalmazhat. Külön figyelmet kapott az a mindennapi probléma, amikor a szervező már eleve hoz magával ismerősöket: a "Foglalt helyek száma" funkcióval az alkalmazáson kívüli résztvevők számára is előre le lehet foglalni a helyet, így elkerülhető a túljelentkezés.

A helyszín megadása interaktív módon történik. A felhasználó választhatja a "Jelenlegi helyzet" lekérését vagy használhatja a beépített Google Places API alapú intelligens szövegkeresőt (amely elég egy közterület vagy intézmény nevét beírni, és azonnal címmé/koordinátává alakítja azt).

Helyszín

Helyszín keresése *

Aranyhal étterem Keresés

Add meg a helyszín nevét, majd kattints a Keresés gombbal

A pontosításhoz használd a keresőt vagy a jelenlegi helyzet gombot!

vagy

Jelenlegi helyzetem használata

Válassz a találatok közül (3) ×

- 📍 Szeged, Kálvária tér 6, 6726 Magyarország
- 📍 Győr, Ménfői út 83, 9019 Magyarország
- 📍 Budapest, Thököly út 121, 1146 Magyarország

5.2 ábra: Helyszín megadása

5.3. Interaktív eseménykeresés és térképi böngészés

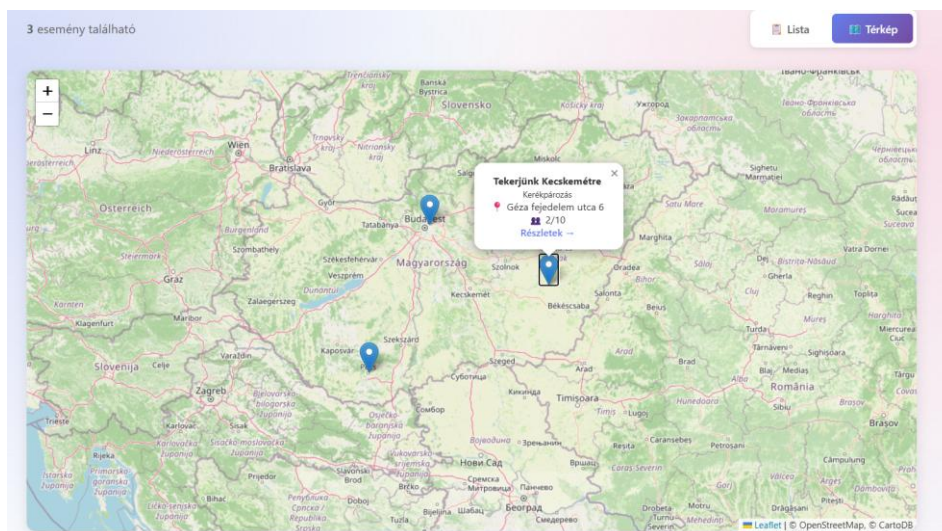
Az alkalmazás központi eleme a /events végpontra elérhető Események oldal, ahol a felhasználók böngészhetnek az elérhető sportolási lehetőségek között. A felső részén egy robusztus szűrőpanel kapott helyet, amellyel keresni lehet szabadszövegesen, vagy szűrni sportágra, nehézségre, státuszra, típusra.

A helyalapú keresést a "Keresés a környékemen" funkció teszi lehetővé. Ennek aktiválásakor a felhasználó profilján beállított "Alapértelmezett helyszín"-t veszi a rendszer alapul és

megjelenik egy távolság-csúszka (1-től 50 km-ig). A rendszer aszinkron módon, az oldal újratöltése nélkül frissíti a találatokat a backend Haversine-formula alapú számításai alapján.

A megjelenítés módja rugalmasan, egy kapcsoló segítségével váltható:

- Lista nézet: Részletes kártyák jelennek meg. A kártya tetején a Leaflet térképkocka jelenik meg a helyszín pontos markerezésével. A kártyákon jól láthatóan megjelenik az ár, a státusz, a sportág, a nehézség, az időpont, a helyszín, résztvevők száma.
- Térkép nézet: Egy nagyméretű, interaktív térképen jelenik meg az összes szűrt találat. A markerekre kattintva egy felugró ablak mutatja az esemény legfontosabb adatait, és innen egy kattintással át lehet navigálni a részletes nézetre.



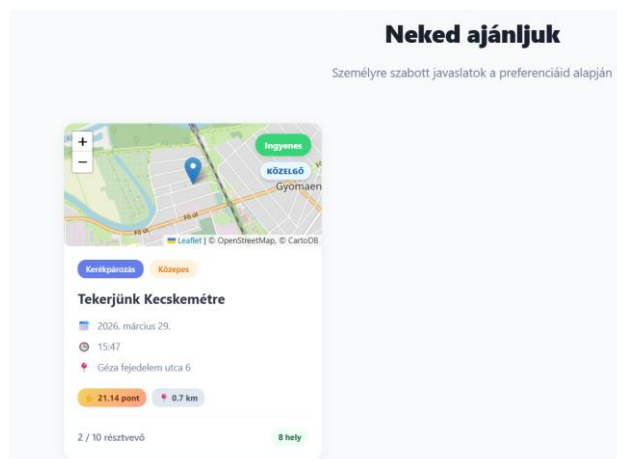
5.3 ábra: Térképes nézet

5.4. Az ajánlórendszer működése és a javaslatok generálása a felületen

A felhasználók a főoldalon egy személyre szabott, dinamikus felülettel találkoznak. Az oldal alján található "Neked ajánljuk" szekció bemutatja az egyedi ajánló algoritmus végeredményét.

Ide azok az események kerülnek be, amelyeket a szerveroldali logika a legrelevánsabbnak ítélt a felhasználó profilja alapján. A transzparencia és a bizalom növelése érdekében a felület vizuálisan is kiemeli, hogy miért ezek az események jelennek meg: a kártyákon egy jól látható jelvényen szerepel a szerver által kiszámított Ajánlási pontszám (pl. "★ 12.5 pont") és a profilban beállított alapértelmezett helyszíntől mért távolság.

Amennyiben a bejelentkezett felhasználónak még nincsenek rögzített preferenciái, vagy az ajánló algoritmus nem talál a profiljához illeszkedő, aktív eseményt a környéken, a felület letisztult marad. Ilyenkor a főoldalon csupán a platform funkcióit bemutató nyitóképernyő jelenik meg, elkerülve a felhasználó számára érdektelen vagy irreleváns tartalom megjelenítését. Ezzel szemben a be nem jelentkezett felhasználók a nyitóképernyőn lejjebb görgetve láthatnak egy ízelítőt a platform publikus, „Közelgő eseményeiből”.



5.4 ábra: Felhasználónak ajánlott események

5.5. Csatlakozás az eseményekhez, értesítési rendszer és értékelés

A platform közösségi élményének alapját a felhasználók közötti interakciók adják. A szoftver egy transzparens, állapotgépre épülő jelentkezési folyamattal, egy aszinkron értesítési rendszerrel, valamint egy utólagos értékelési modullal támogatja a zökkenőmentes eseményszervezést.

A jelentkezés menete és a résztvevők kezelése: Amikor egy felhasználó az esemény részleteinél a „Csatlakozás” gombra kattint, egy felugró űrlap jelenik meg. Itt a sportoló opcionálisan megadhatja, hogy hány extra vendéget (alkalmazáson kívüli barátot) hoz magával, illetve rövid szöveges üzenetet hagyhat a szervezőnek. A jelentkezés kimenetele az esemény beállításaitól függ:

- Ha a szervező nem kért előzetes jóváhagyást, a jelentkezés azonnal *Megerősítve* (*Confirmed*) státuszba kerül. A felület azonnal frissíti a szabad helyek számát, illetve megjeleníti a jelentkezőt a Résztvevők között.

- Ha az esemény jóváhagyáshoz kötött, a jelentkezés *Függőben (Pending)* státuszt kap. Ekkor a szervező a “Saját események” fül alatt a “Jóváhagyások”-nál illetve a létrehozott esemény adminisztrációs felületén egy listában látja a jelentkezőket, akiket egyetlen kattintással elfogadhat vagy elutasíthat.

5.5 ábra: Eseményre való jelentkezés illetve részletes nézet

A beépített értesítési rendszer: A tranzakciókról és státuszváltozásokról a rendszer automatikusan belső és email-es értesítéseket generál, amelyek a felső navigációs sávban található harang ikonra kattintva érhetők el. A rendszer az alábbi főbb eseményekről küld azonnali visszajelzést:

- Új jelentkezés érkezett (a szervező kapja, ha valaki csatlakozni szeretne).
- Jelentkezés elbírálása (a jelentkező kapja, ha a szervező elfogadta vagy elutasította a kérelmét).
- Ajánlott esemény (a sportoló kapja, ha egy új, a preferenciáihoz és a helyzetéhez tökéletesen illeszkedő esemény jött létre a környéken).

Utólagos értékelések és bizalomépítés: Az amatőr sport közösségekben kiemelt szerepe van a megbízhatóságnak. Ennek elősegítése érdekében a szoftver tartalmaz egy értékelési modult. Amint egy esemény elindul és státusza automatikusan *Folyamatban (Ongoing)* állapotba vált, a felület engedélyezi az értékelést. A megerősített résztvevők egy 1-től 5 csillagig terjedő skálán minősíthetik a szervezést, valamint szöveges visszajelzést is írhatnak.

6. Tesztelés és kiértékelés

6.1. Funkcionális és felhasználói elfogadási tesztelés

A rendszer validálása Use-case alapú, manuális teszteléssel történt. Ennek során végigiteráltam a szoftver összes főbb funkcióján, szimulálva a valós felhasználói viselkedést. A tesztelés kiterjedt a hibás adatok megadására (például rossz formátumú email cím, jelszó-eltérések, múltbeli dátumok megadása eseményhez), amely során megbizonyosodtam arról, hogy a frontend validációk és a backend hibaüzenetek megfelelően tájékoztatják a felhasználót a felületen a beépített *ToastService* segítségével.

Az objektív kiértékelés érdekében a fejlesztés végén egy zártkörű, feketedobozos béta tesztelést is szerveztem bevonva a célközönséget reprezentáló ismerősöket. A tesztalanyok különböző eszközökön (asztali számítógép, laptop, okostelefon) és böngészőkben (Chrome, Safari, Firefox) próbálták ki az alkalmazást, a háttérkód ismerete nélkül. Feladatuk az alkalmazás teljes életciklusának – regisztráció, preferenciák megadása, új esemény létrehozása, csatlakozás, majd utólagos értékelés – végrehajtása volt.

A tesztelés során feltárt főbb hibák és azok javítása: A feketedobozos és keresztplatformos tesztelés rávilágított néhány kritikus felhasználói hiányosságra:

1. Helyszínválasztási anomália (UX hiba): Az új események létrehozásánál a tesztalanyok számára nem volt egyértelmű, hogy a „Helyszín neve” mező kitöltése önmagában nem elegendő; a háttérrendszernek szüksége van a Google Places API által visszaadott pontos földrajzi koordinátákra is. Ennek orvoslására egyértelműbbé tettem a lépéseket, illetve szigorúbb űrlap-validációt (a koordináták kötelezővé tételét) vezettem be.
2. Reszponzivitási problémák (UI hiba): A kisebb ablakban történő tesztelés során kiderült, hogy bizonyos komponensek nem alkalmazkodnak megfelelően a képernyőméretekhez. A komplexebb űrlapoknál és a térkép konténerénél vízszintes túlcsozdulás lépett fel, illetve a felső navigációs sáv elemei egymásra csúsztak, kitakarva fontos gombokat. A hibát CSS Média lekérdezések finomhangolásával, a flexbox és grid struktúrák mobilnézetre történő optimalizálásával javítottam.

Ezen finomhangolások bevezetése után az ismételt tesztkörök már fennakadások nélkül futottak le. A feketedobozos tesztelés főbb lépéseit és eredményeit az alábbi felhasználói tesztnapló foglalja össze.

Teszt azonosító	Tesztelt funkció	Elvárt működés	Tapasztalt eredmény / Visszajelzés	Státusz / Megoldás
TC-01	Regisztráció és belépés	A felhasználó sikeresen fiókot hoz létre és bejelentkezik.	Hibátlan működés, a validációs hibaüzenetek (pl. foglalt email) egyértelműek.	Sikeres
TC-02	Preferenciák beállítása	A profilban megadható a sportág, a tudásszint és az érdeklődés.	A felület gyors, az adatok azonnal mentésre kerülnek az adatbázisban.	Sikeres
TC-03	Új esemény létrehozása	Űrlap kitöltése és mentése térképes helyszínmegadással.	Hiba (UX): A tesztelők kihagyták a térképes keresést, így koordináták hiányában a mentés elszállt.	Javítva: Kötelező keresőhasználat és új státuszjelző bevezetése.
TC-04	Reszponzív megjelenés	A felület (kisebb kijelzőn) is jól használható marad.	Hiba (UI): Az űrlapok kilógtak a képernyőről, a navigációs sáv elemei egymásra csúsztak.	Javítva: CSS media query-k és hamburger menü optimalizálása.
TC-05	Csatlakozás eseményhez	Jelentkezés elküldése egy másik felhasználó eseményére.	Az űrlap megfelelően kezelte a plusz vendégek számát, a gomb inaktívvá vált a mentés alatt.	Sikeres
TC-06	Szervezői jóváhagyás	A szervező elfogadja a beérkezett jelentkezést.	A státusz azonnal frissült, a létszámkorlát <i>is_full</i> dinamikusán követte a változást.	Sikeres
TC-07	Belső értesítések	Státuszváltozáskor a felek aszinkron értesítést kapnak.	A "harang" ikon jelezte az új eseményeket (jelentkezés, elfogadás), a lista helyesen töltődött be.	Sikeres

TC-08	Események értékelése	Lezárt esemény után a résztvevők 1-5 csillagos értékelést adhatnak.	A jogosultságkezelés működött (csak résztvevő értékelhet), az átlagpontszám azonnal frissült a profilban.	Sikeres
-------	----------------------	---	---	---------

6.1 ábra: Felhasználói tesztnapló

6.2. A geolokációs és ajánló algoritmus pontosságának vizsgálata

A szoftver két legkomplexebb logikai moduljának, a térképes szűrésnek és a személyre szabott ajánlórendszernek a tesztelése kiemelt figyelmet kapott.

Geolokációs tesztek: A távolságalapú szűrés tesztelése során előre definiált koordinátákkal hoztam létre eseményeket (például Szegeden és Budapesten). A felhasználói profilban a keresési sugarat 10, majd 50 kilométerre állítva vizsgáltam a backend *calculate_distance* függvényének működését. A tesztek igazolták, hogy a rendszer pontosan szűri ki a sugarat meghaladó eseményeket. A Google Places API integrációja is megbízhatóan működött: a szabad szavas helyszíneresések (pl. egy sulis tornaterme vagy egy park neve) sikeresen alakultak át pontos GPS koordinátákká, amelyek azonnal megjelentek a Leaflet térképen.

Az ajánlórendszer kiértékelése: A pontozásos (*recommendation_score*) algoritmus validálásához különböző profilú tesztfelhasználókat hoztam létre az adatbázisban ("Kezdő futó" és "Profi kosaras"). A tesztek során az algoritmus a vártnak megfelelően rangsorolt:

- A preferenciák (1-10 közötti érdeklődési szint) és a tudásszint alapján egyértelműen a releváns sportágak kerültek a lista elejére.
- Sikeresen működött a teltségi szint bónusz pontozása is: a majdnem betelt eseményeket a rendszer előrébb sorolta.
- A részvételi előzmények tesztelésekor látható volt, hogy egy esemény pozitív (4-5 csillagos) értékelése után a rendszer hajlamosabb volt hasonló sportágat ajánlani a későbbiekben.

6.3. Ismert korlátok és továbbfejlesztési lehetőségek

Bár a bemutatott webalkalmazás maradéktalanul teljesíti a feladatkiírásban megfogalmazott célokat és egy stabil, jól használható alapvető szintet képvisel, a tervezés és tesztelés során azonosításra került néhány korlát, illetve lehetőség a jövőbeli továbbfejlesztésre:

1. Automatizált tesztelés hiánya: Jelenleg a rendszer nem rendelkezik automatizált (unit és integrációs) tesztkészlettel. Bár a manuális tesztelés a jelenlegi fázisban elegendő volt, egy esetleges éles, nagy felhasználószámú üzemeltetés előtt elengedhetetlen a backend (pl. PyTest használatával) és a frontend (pl. Jest és Cypress) automatizált tesztjeinek megírása a regressziós hibák elkerülése végett.
2. Képfeltöltési funkció befejezése: Ahogy a szoftvertervezési fejezetben is említésre került, az eseményekhez tartozó képgalériák (*EventImage*) adatbázis-modellje és a backend API végpontjai már elkészültek. Ennek teljes körű kliensoldali implementációja, beleértve a képvágó és optimalizáló modulokat, a következő fejlesztési iteráció feladata.
3. Valós idejű kommunikáció: Bár a rendszer rendelkezik aszinkron adatbázis-alapú értesítésekkel, a csapattársak közötti kommunikációt nagyban megkönnyítené egy eseményspecifikus, valós idejű chat szoba bevezetése. Ennek technológiai alapja a Django Channels integrációja lehetne.
4. Skálázhatóság és felhőszolgáltatások: A Google Maps API használata nagyszámú felhasználó esetén költségessé válhat, így a későbbiekben érdemes lehet vizsgálni a geokódolási kérések szerveroldali gyorsítótárazását (Caching, pl. Redis segítségével), vagy ingyenes alternatívák (pl. OpenStreetMap Nominatim API) bevonását.

Irodalomjegyzék

- [1] **Angular hivatalos dokumentáció** Utolsó megtekintés (2026. 04. 06.) <https://angular.dev/>
- [2] **TypeScript Handbook dokumentáció** Utolsó megtekintés (2026. 04. 06.)
<https://www.typescriptlang.org/docs/handbook/intro.html>
- [3] **MDN Web Docs – HTML5, CSS3 és JavaScript referenciák** Utolsó megtekintés (2026. 03. 29.)
<https://developer.mozilla.org/>
- [4] **Django web framework hivatalos dokumentáció** Utolsó megtekintés (2026. 04. 08.)
<https://docs.djangoproject.com/>
- [5] **Django REST Framework dokumentáció** Utolsó megtekintés (2026. 04. 08.) <https://www.django-rest-framework.org/>
- [6] **PostgreSQL hivatalos dokumentáció** Utolsó megtekintés (2026. 03. 17.) <https://www.postgresql.org/docs/>
- [7] **Leaflet dokumentáció – JavaScript library for interactive maps** Utolsó megtekintés (2026. 04. 10.)
<https://leafletjs.com/reference.html>
- [8] **Google Maps Platform dokumentáció – Places API** Utolsó megtekintés (2026. 04. 16.)
<https://developers.google.com/maps/documentation/places/web-service/overview>
- [9] **Google Maps Platform dokumentáció – Geocoding API** Utolsó megtekintés (2026. 04. 16.)
<https://developers.google.com/maps/documentation/geocoding/overview>
- [10] **Gillespie, R. W. (1975). *The Haversine Formula*** Utolsó megtekintés (2026. 04. 07.)
<https://community.esri.com/t5/coordinate-reference-systems-blog/distance-on-a-sphere-the-haversine-formula/ba-p/902128>

Nyilatkozat

Alulírott Rák Dávid, programtervező informatikus szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Számítástudomány Alapjai Tanszékén készítettem, Programtervező informatikus BSC diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel.

Tudomásul veszem, hogy szakdolgozatomat / diplomamunkámat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

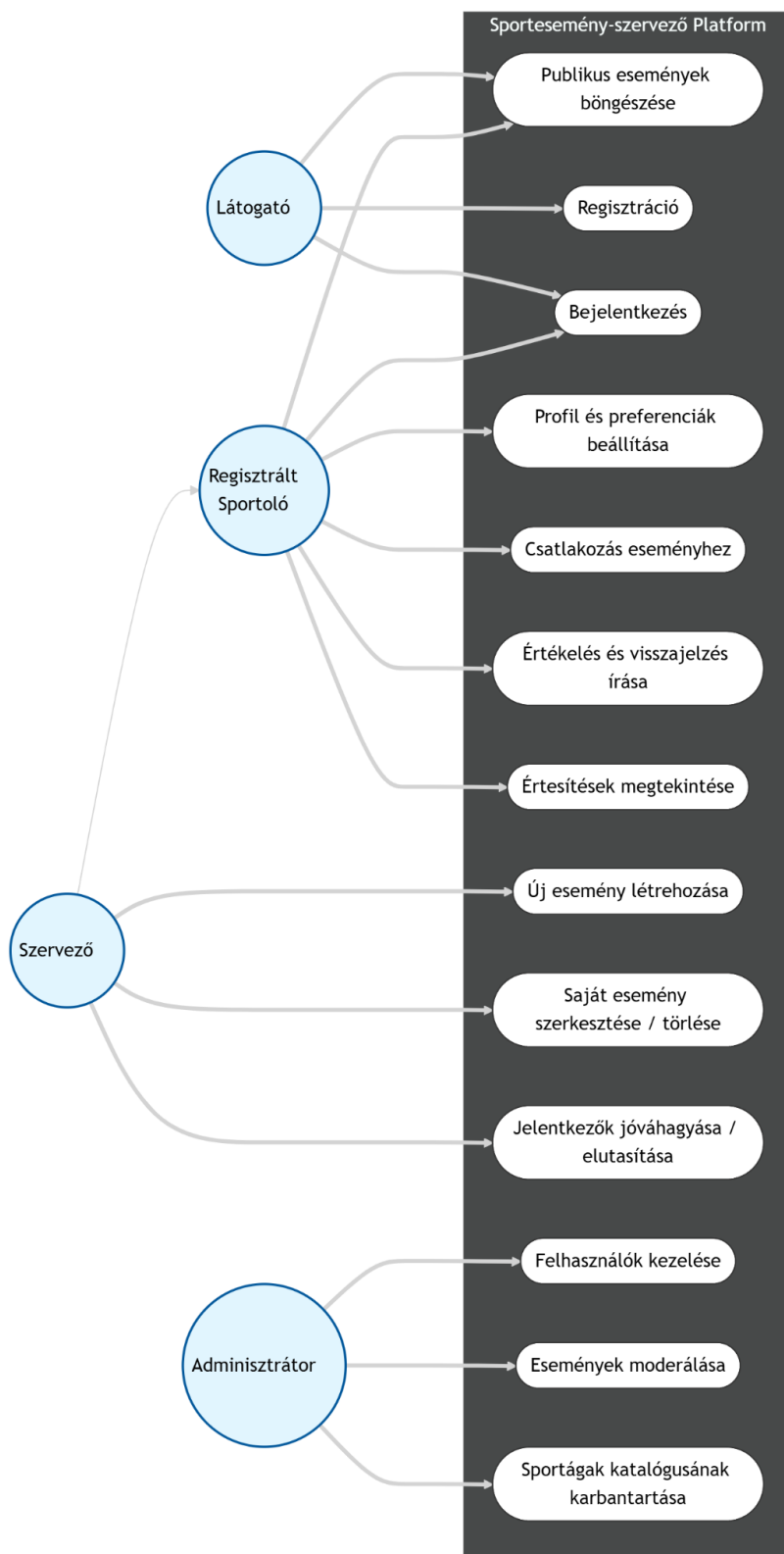
Dátum: 2026.05.16

Aláírás

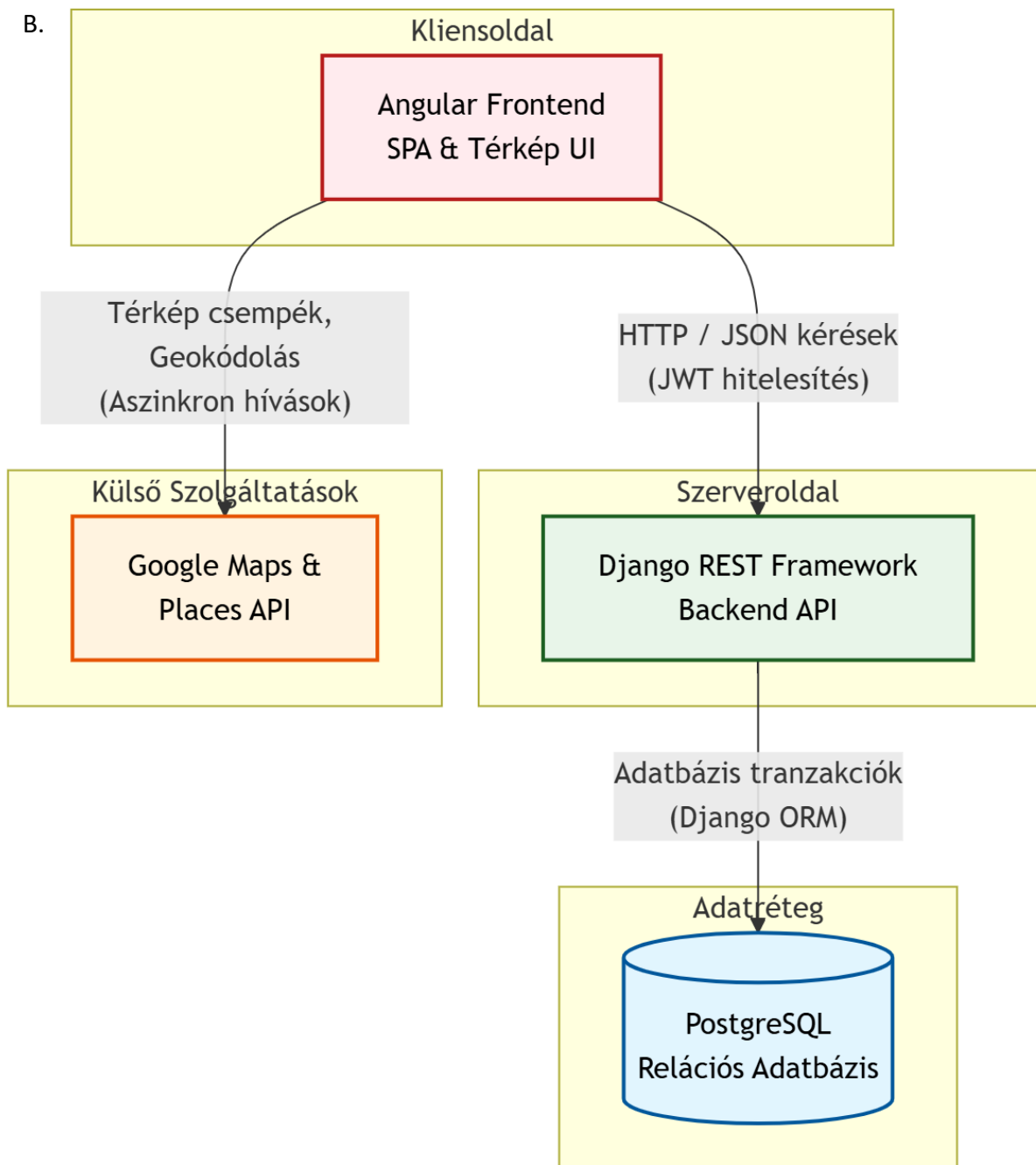
Rák Dávid

Mellékletek

A.

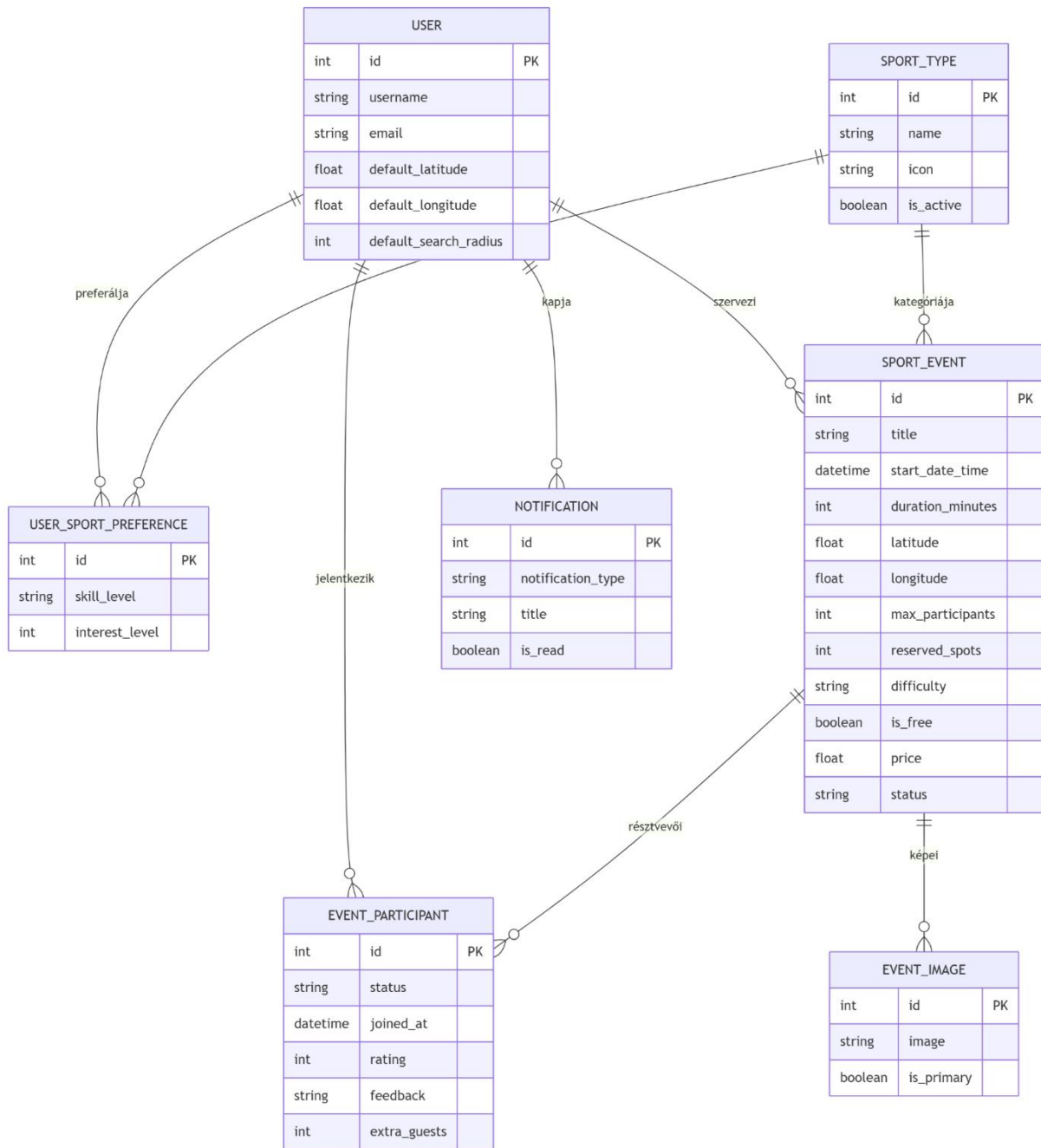


Használati eset diagram (3.2)



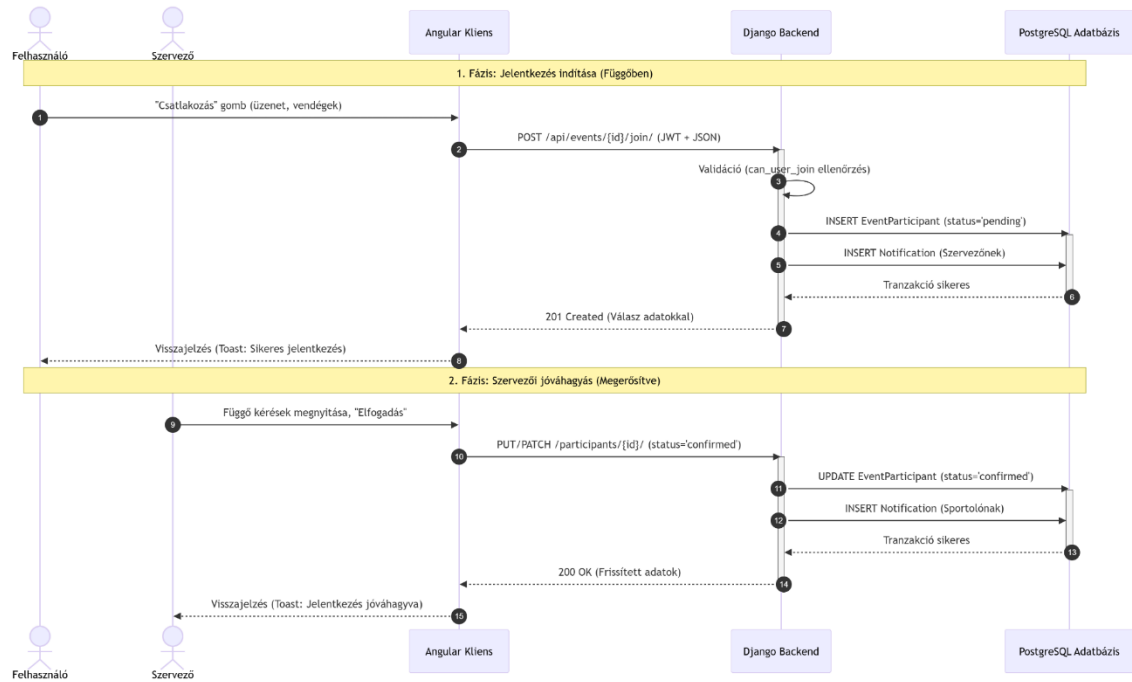
Komponens diagram (3.3)

C.



Egyed-kapcsolat diagram (3.4)

D.



Szekvencia diagram (3.5)